

## **Relatório – Parte I**

### **Decodificador de instruções hexadecimais RISC-V**

#### **1.0. Introdução**

Este relatório consiste na implementação de um decodificador de instruções hexadecimais ou binárias da arquitetura RISC-V. O programa recebe um arquivo de instruções (ROM) e retorna sua instrução decodificada em linguagem assembly e os valores específicos de cada campo da instrução.

#### **2.0. Objetivos**

Os objetivos, baseados na descrição do trabalho foram:

- \* Implementar um decodificador de instruções RISC-V
- \* Identificar o formato de cada instrução
- \* Identificar o mnemônico (operação) correspondente
- \* Extrair todos os campos e utiliza-los respectivamente
- \* Calcular o valor PC de cada instrução
- \* Mostrar a instrução reconstruída, junto de seu tipo e valores de seus campos
- \* Contabilizar cada instrução por formato e descrever seu CPI médio ao final

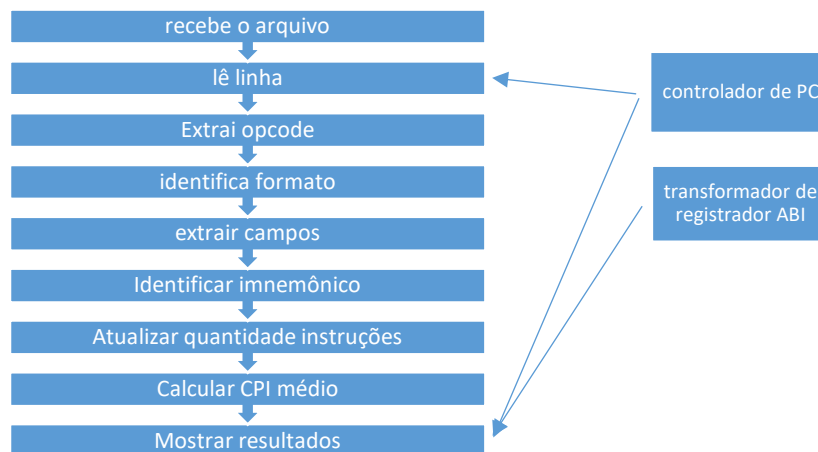
#### **3.0. Metodologia**

A metodologia foi feita com o modelo incremental do desenvolvimento de software em mente. Ou seja, o projeto fora dividido em partes menores que incrementavam a implementação a cada etapa. A tabela fora dividida entre etapas consecutivas que iriam levar ao caminho final, junto de etapas extras que estão espalhadas por outras etapas e servem de refinamento e/ou característica fundamental de certa parte da implementação.

O diagrama consiste de partes respectivas que derivam (em sua maior parte) das anteriores. Além disto a implementação ira de necessitar de dois incrementos extras: um controlado do endereço de PC e um transformador de registrador em nomenclatura ABI; que são fora da estrutura normal do código, e servem de complemento para algumas partes da metodologia.

A estrutura é descrita na figura 3.1:

### 3.1. Estrutura normal, mais etapas extras.



## 4.0. Implementação

### 4.1. Leitura das instruções:

Os dados entram no programa através de um arquivo “rom.txt” dedicado na pasta output do projeto. Como especificado no projeto, deve-se possuir apenas uma instrução por linha.

A instrução então é lida no primeiro arquivo dos incrementos: “file-interpreter”, utilizando uma função “lineReader()” dedicada que depende da variável de arquivo “ifstream” que vem da biblioteca <fstream> de c++. Esta variável é estática pois o arquivo irá (na maioria das vezes) possuir mais de uma linha, logo não desejamos ter de reabrir o arquivo toda vez que necessitamos ler a próxima linha do arquivo.

Então, após extrair a linha utilizando a instrução getline(), devolvemos ela ao usuário, tratando-a como binário se for previamente hexadecimal, caso já for binária e não possuir erros de estrutura, irá ser devolvida imediatamente como uma string.

### 4.2 Decodificação:

A decodificação é feita em etapas: determinar o tipo de instrução, determinar o mnemônico, determinar registradores ABI, valor de PC e, por fim, o CPI médio por instrução.

Determinar o tipo de instrução:

No arquivo “instruction-type”, recebemos o binário da instrução anterior e então extraímos seu valor Opcode (bits 0-6), que é o único valor necessário para descobrir o tipo da instrução. Após isto possuímos 6 funções que representam respectivamente cada um dos tipos possíveis de instrução. Baseado no Opcode, entregamos ao parâmetro da função desejada um array de strings que irá receber cada um dos binários divididos dependendo da estrutura da divisão do tipo.

Determinar o mnemônico:

O mnemônico é extraído baseado em uma até três destas informações (dependendo do tipo de instrução): Opcode, funct3 e funct7. Baseado na documentação do RISC-V, basta fazer uma logica combinacional que retorne o mnemônico baseado em um destes três tipos.

Determinar registradores ABI:

Igual a etapa anterior, é feito uma simples lógica baseada na documentação do RISC-V para transformar o valor de 0 a 31 dos registradores no seu tipo padrão ABI, esta etapa só é utilizada na demonstração da instrução decodificada.

Valor de PC:

O registrador PC recebe um arquivo dedicado que, com apenas uma variável inteira estática, é encapsulada unicamente ao arquivo e pode apenas ser alterada baseada nas funções presentes no mesmo.

CPI médio por instrução:

Então, por último, esta etapa extra conta o tipo de instrução ao ser lida no arquivo de demonstração da decodificação, e então faz a soma das instruções totais do arquivo e divide ela utilizando a formula:  $100 * CPI / TOTAL \text{ DE INSTRUÇÕES}$ .

## 5.0. Analise dos resultados

Ao fim, em um arquivo dedicado de demonstração, ele possui duas funções, uma que mostra o tipo de arquivo, seu valor PC, seus campos e valores e a instrução hexadecimal junto da decodificada em assembly. A outra função nos mostra quantas vezes cada tipo de instrução apareceu no arquivo e qual a sua porcentagem de aparecimento.

Deve-se parecer algo assim para cada instrução:

---

**PC: 4**

---

**Format: R**

**Mnemonic: add**

**RD: 5**

**RS1: 6**

**RS2: 7**

**FUNCT3: 000**

**FUNCT7: 0000000**

---

**Hexadecimal instruction:**

---

**0x007302B3**

---

**Assembly instruction:**

---

**add t0, t1, t2**

---

E após a demonstração de todos os tipos, o programa irá mostrar a quantidade de vezes que cada operação apareceu e sua porcentagem de aparecimento no arquivo.

Deve-se parecer algo assim:

---

**Instruction type count:**

---

**R: 2**

**I: 4**

**S: 1**

**B: 1**

**J: 1**

**U: 1**

---

**Avarage percentage per type:**

---

**R: 20%**

**I: 40%**

**S: 10%**

**B: 10%**

**J: 10%**

**U: 10%**

---

## **6.0. Conclusão**

Ao fim, o decodificador funciona perfeitamente e consegue imprimir quaisquer tipos de instrução hexadecimal ou binária, além de possuir validações para linhas má escritas, espaços em branco ou comentários no arquivo ROM. O programa não possui implementação para detecção de pseudo-instruções, porém é algo que vira com o futuro da implementação na parte II.

## **7.0. Referencias**

<https://www.cs.cornell.edu/courses/cs3410/2025fa/rsrc/riscv-instructions-2.html>

<https://www.eg.bucknell.edu/~csci206/riscv-converter/>

<https://docs.riscv.org/reference/abi/v1.0/riscv-cc-register-convention.html>